



← Back to student guides

[📈] MLflow

Tracking & registry

3 levels

111 sections



The Complete Guide

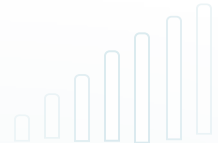
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

RUN

MODEL

REGISTRY

SERVE



Quick Start

A one-page starter for this tool. Scroll through it here, or open it full screen.

Open PDF







15

sections

32

examples

This is part one of three. It covers **everything you need to do real work with MLflow**, not a teaser. By the end you can track an experiment without restructuring your code, log parameters, metrics, and artifacts on purpose, compare thirty runs in a table and a chart, package a model so someone else can load it without knowing your framework, register and promote it through stages, and serve it behind an HTTP endpoint. Mid-level and Senior take the same topics further; nothing here is thrown away.

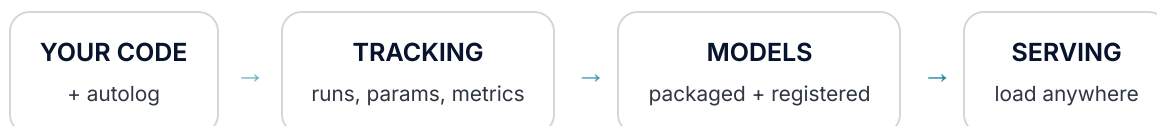
Each section ends with a **Try it** task. Do them as you go. They take a few minutes each, and these ideas only stick once you have watched a metric appear in the MLflow UI while your script is still running.

What MLflow is, and the problem it solves

You trained a model last Tuesday. It scored 0.94 AUC. Today you cannot reproduce it. The notebook has moved on, the learning rate in the cell is not the one that ran, the feature list changed, and the only surviving record of 0.94 is a number in a Slack message.

Carelessness was never the problem. A system was missing: **nothing was writing anything down**. Every fact about that run lived in your terminal scrollbar and your memory.

MLflow writes it down. You add a handful of lines, often just one, and it records the parameters, the metrics over time, the artifacts, the source file, the git commit, and the environment needed to load the model again, into a store you and your team can query.





a bare pickle from someone's laptop. Anything that understands that contract can load it: a batch job, a REST server, a Spark UDF, a Docker image. That single property is what turns a training script into something deployable.

That gives you four components, and they map exactly onto four problems:



Tracking

Runs with parameters, metrics, tags, and artifacts. The answer to "what did we try and what happened?"



Models

A packaging format with flavours, a signature, and an environment. The answer to "how do I load this?"



Model Registry

Named models with versions, aliases, and stage transitions. The answer to "which one is in production?"



Projects

A declared entry point and environment so a run is reproducible by command. The answer to "how do I rerun it?"

The vocabulary is small, and getting it straight now prevents most later confusion:

Term	What it is
Run	One execution: parameters, metrics, tags, artifacts, and a status. The atomic unit
Experiment	A named container of runs. The unit you compare within
Tracking server	The service that stores run metadata and serves the UI
Backend store	Where run metadata lives: a database, or local files
Artifact store	Where files live: S3, GCS, Azure, or a local directory
Artifact	Any file attached to a run: a plot, a CSV, a model directory



Flavour	How a logged model can be loaded. <code>sklearn</code> , <code>pytorch</code> , <code>pyfunc</code> , and others
Signature	The declared input and output schema of a model
Registered model	A named entry in the registry with numbered versions

You need little to follow along: Python, a script that trains something small, and a folder. A tiny dataset and a five-line model are the best place to start, because a broken run costs you nothing.

► TRY IT

1. Find your most recent training script. Write down, from memory, the exact learning rate and feature set it last ran with.
2. Check the script and see whether you were right.
3. Find the model file it produced and try to name the git commit that trained it.
4. Count the files on your disk matching `model_final*` , `*_v2*` , or a date in the name.

Expected result – at least one of those is unanswerable, and usually all three. The third one, tying a model file to a commit, is the question that turns into an afternoon of archaeology during a code review, and it is the one MLflow answers with a run id.

The four components, and the two stores behind them

MLflow is often described as one tool. It is four that share a store, and you can adopt them in order.



1 Tracking

Log params, metrics, and artifacts. Useful on day one, with no infrastructure.



3 Registry

Promote a specific model version by alias, so consumers stop hard-coding paths.

4 Serving / Projects

Load that version in a batch job, a REST server, or a container.

Two stores sit behind everything, and confusing them causes a whole category of errors:

YOUR PROCESS

`mlflow client`

`log_param · log_metric · log_model`

Autolog patches

sklearn, torch, xgboost: params and metrics for free



TRACKING SERVER: TWO SEPARATE STORES

Backend store

Runs, params, metrics, tags, and the registry . Needs a database for aliases

Artifact store

Files: models, plots, reports. S3, GCS, Azure, or a directory

Web UI

Reads both. Compare, filter, chart





Moveable pointer. Promotion is a tag move

Consumers

Batch job · REST server · container · Spark UDF

The detail that surprises people: by default the client reads and writes the `artifact store directly`, and the tracking server only hands out a URI. That is why metrics can appear in the UI while artifacts 404, and it means every user and CI job needs storage credentials unless you enable proxied access.

Store	Holds	Configured by
Backend store	Runs, params, metrics, tags, experiment and registry metadata	<code>--backend-store-uri</code>
Artifact store	Files: models, plots, CSVs, anything you log as an artifact	<code>--default-artifact-root</code>

BASH

Copy

```
# Local files only – no server, no database. Creates ./mlruns
python train.py

# A local server with a SQLite backend – needed for the Model Registry
mlflow server --backend-store-uri sqlite:///mlflow.db \
              --default-artifact-root ./mlartifacts \
              --host 127.0.0.1 --port 5000

# A team setup: Postgres for metadata, S3 for files
mlflow server --backend-store-uri postgresql://user:pass@host/mlflow \
              --default-artifact-root s3://my-bucket/mlflow \
              --host 0.0.0.0 --port 5000
```

The Model Registry requires a database backend

With the default file store, tracking works and the registry does not. You get a clear error the first time you call `register_model`. SQLite is enough for one person; Postgres or



Your code points at a tracking server with one line, or one environment variable:

PYTHON

Copy

```
import mlflow
mlflow.set_tracking_uri("http://127.0.0.1:5000") # or "sqlite:///mlflow.db", or a path
mlflow.set_experiment("churn-baseline")
```

BASH

Copy

```
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

► TRY IT

1. Run `pip install mlflow`, then `mlflow server --backend-store-uri sqlite:///mlflow.db --default-artifact-root ./mlartifacts --port 5000`.
2. Open `http://127.0.0.1:5000` and note that it is empty: no runs, no experiments except `Default`.
3. Sketch the two stores from memory and label which one a metric goes to and which one a plot goes to.
4. Now run a script with no `set_tracking_uri` at all and find the `mlruns/` directory it created.

Expected result – step four is the one worth doing: MLflow works with zero configuration by writing to a local folder, which is why people accidentally end up with runs scattered across five `mlruns` directories. Knowing that the default exists is how you avoid it.

Your first tracked run, line by line

Start from a script with no MLflow in it:



```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

n_estimators, max_depth = 200, 5

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth, random_state=42)
model.fit(X_train, y_train)
print("auc", roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))
```

Here it is tracked:

train.py (after)

Copy

```
import mlflow
import mlflow.sklearn
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("churn-baseline")

params = {"n_estimators": 200, "max_depth": 5, "random_state": 42}

with mlflow.start_run(run_name="rf baseline") as run:
    mlflow.log_params(params)

    X, y = load_breast_cancer(return_X_y=True, as_frame=True)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = RandomForestClassifier(**params).fit(X_train, y_train)
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])

    mlflow.log_metric("auc", auc)
    mlflow.sklearn.log_model(model, name="model", input_example=X_train.head(3))
```



Run it, then open the UI. Four things happened:

01 A run was created

With a run id, an experiment, a status, a start and end time, and the user and source file that produced it.

02 Parameters were recorded

Immutable strings. A parameter is set once and describes the configuration of the run.

03 A metric was recorded

A float with a step and a timestamp, so a metric is a series rather than a single number.

04 A model was logged

Not a bare pickle, but a directory with an `MLmodel` file declaring its flavours, its signature, and its dependencies.

Use the context manager, always

`with mlflow.start_run():` is the correct default. A bare `mlflow.start_run()` followed by `mlflow.end_run()` works until something raises in between, and then you have a run that claims to be running days later. In a notebook, an un-ended run also silently captures your next cell's logs.

► TRY IT

1. Run the tracked script and open the run in the UI. Visit each tab: Overview, Parameters, Metrics, Artifacts.
2. Under Artifacts, open `model/MLmodel` and read it. Note the flavours and the signature.



Expected result – the `MLmodel` file is the thing to read. It is the entire "why MLflow" argument in twenty lines: a declared loader, a declared schema, and a declared environment, which is what makes the artifact portable rather than personal.

Params, metrics, tags, artifacts: choosing correctly

Four things you log, with four different meanings. Choosing correctly is most of what separates a readable experiment table from an unusable one.

PYTHON

Copy

```
# Parameters: configuration. Immutable, stored as strings.
mlflow.log_param("learning_rate", 3e-4)
mlflow.log_params({"batch_size": 64, "optimizer": "adamw", "features": "v3"})

# Metrics: numbers that can move. Float, with an optional step.
mlflow.log_metric("train_loss", 0.412, step=epoch)
mlflow.log_metric("val_loss", 0.503, step=epoch)
mlflow.log_metrics({"precision": 0.91, "recall": 0.88}, step=epoch)

# Tags: mutable, searchable labels for organisation.
mlflow.set_tag("stage", "baseline")
mlflow.set_tags({"team": "risk", "dataset": "v3", "owner": "amina"})

# Artifacts: files.
mlflow.log_artifact("outputs/confusion_matrix.png")
mlflow.log_artifact("outputs/report.html", artifact_path="reports")
mlflow.log_artifacts("outputs/plots", artifact_path="plots") # a whole directory

# Convenience helpers that avoid touching disk yourself
mlflow.log_figure(fig, "plots/roc.png")
mlflow.log_table(predictions_df, "predictions.json")
```



Kind	Mutable	Type	Use for
Param	No: logging twice with a different value errors	String	Configuration: hyperparameters, feature-set version, seed
Metric	Yes: appends a new step	Float	Anything measured: loss, AUC, latency, training minutes
Tag	Yes: overwrites	String	Organisation and search: team, purpose, ticket, dataset
Artifact	Append-only	File	Plots, reports, predictions, the model itself

Two rules resolve nearly every "which one?" question. **If it is a number you might plot, it is a metric. If it is something you might change your mind about after the run, it is a tag.**

Params are immutable, and that bites in loops

Calling `log_param("lr", x)` twice with different values raises. It is deliberate, a run has one configuration, but it surprises people who log parameters inside a training loop or a retry. Log parameters once, at the top of the run, and use metrics for anything that varies during it.

► TRY IT

1. Log the same parameter twice with different values and read the error.
2. Log a metric with `step=0..9` in a loop and look at the Metrics tab, and note that you get a chart, not a number.
3. Set a tag, then set the same tag to a different value, and confirm it overwrites.
4. Log a matplotlib figure with `log_figure` and confirm it appears under Artifacts without you writing a file.

Expected result – the immutability difference in steps one and three is the whole model: params describe the run, tags describe how you think about the run, and



PYTHON

Copy

```
import mlflow

mlflow.autolog()           # everything MLflow can detect

# or per-framework, which is more predictable
mlflow.sklearn.autolog()
mlflow.pytorch.autolog()
mlflow.tensorflow.autolog()
mlflow.xgboost.autolog()
mlflow.lightgbm.autolog()
mlflow.transformers.autolog()
```

With `sklearn.autolog()` active, calling `.fit()` logs the estimator's parameters, training metrics, the fitted model with a signature, and, for search objects like `GridSearchCV`, child runs for every candidate configuration:

PYTHON

Copy

```
import mlflow
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV

mlflow.set_experiment("gb-sweep")
mlflow.sklearn.autolog(max_tuning_runs=10)

grid = GridSearchCV(
    GradientBoostingClassifier(random_state=42),
    {"max_depth": [2, 3, 4], "learning_rate": [0.05, 0.1]},
```



```
with mlflow.start_run(run_name="gb grid search"):  
    grid.fit(X_train, y_train)           # one parent run, plus child runs per candidate
```

Autologged	Typically includes
Parameters	Every constructor argument of the estimator or the training call
Metrics	Training scores, and per-epoch metrics for deep learning frameworks
Model	Logged with a signature inferred from the training data
Artifacts	Feature importances, metric plots, and framework-specific summaries
Child runs	One per candidate in a hyperparameter search, up to <code>max_tuning_runs</code>
Datasets	For some flavours, a dataset reference describing the training input

Call autolog before you create the estimator

Autologging patches the framework, so it has to be active before the training call, and for some frameworks before the object is constructed. Putting `mlflow.autolog()` at the top of the entry point, next to the imports, removes an entire class of "why did nothing get logged".



1. Delete every logging call from your script, add `mlflow.sklearn.autolog()` , and compare what gets recorded.
2. Run a `GridSearchCV` under autolog and look at the parent run's child runs in the UI.
3. Add one custom metric of your own alongside autolog and confirm both appear.
4. Move the `autolog()` call to

Expected result – after the estimator is created and see what changes.

Expected result – the child-run structure in step two is the payoff: a six-candidate search becomes six comparable runs in one table, for free. Step four teaches you the ordering rule the hard way, which is the way that sticks.

The MLflow Model format: flavours and signatures



PYTHON

Copy

```
import mlflow

# Native flavour: the real object, with its own API
sk_model = mlflow.sklearn.load_model("runs:/a1b2c3d4/model")
sk_model.feature_importances_

# Generic flavour: just predict(). Framework-agnostic.
pyfunc = mlflow.pyfunc.load_model("runs:/a1b2c3d4/model")
pyfunc.predict(X_test)
```

A signature is a contract. It declares the input columns and types and the output type, so a wrong-shaped or wrong-typed request is rejected at the boundary rather than producing nonsense predictions.

PYTHON

Copy



```
signature = infer_signature(X_train, model.predict(X_train))
mlflow.sklearn.log_model(
    model,
    name="model",
    signature=signature,
    input_example=X_train.head(3),      # also enables the UI's sample request
)
```

Piece	Why it matters
<code>flavors</code>	How the model can be loaded, and by whom
<code>signature</code>	Input/output schema: validated at serving time
<code>input_example</code>	A concrete sample, used for docs and for signature inference
<code>requirements.txt</code>	The pinned environment, so serving does not guess
<code>run_id</code>	The link back to the run, and therefore to the params and metrics

Log a signature, or serving will accept anything

Without a signature, an endpoint takes columns in the wrong order or of the wrong type and returns predictions that are wrong. `infer_signature` costs one line and converts a



► TRY IT

1. Log a model, then open its `MModel` file and identify every flavour it declares.
2. Load it twice (once with `mlflow.sklearn.load_model`, once with `mlflow.pyfunc.load_model`) and note what each object can do.
3. Log one model with a signature and one without, then feed both a DataFrame with the columns in the wrong order.
4. Open the `requirements.txt` inside the model directory and compare it against your environment.

Expected result – step three is the demonstration: the unsigned model returns confident nonsense, the signed one refuses. That difference is the argument for signatures, and no amount of reading it is as convincing as seeing both outputs.



`search_runs` returns a pandas DataFrame with flattened `params.*`, `metrics.*`, and `tags.*` columns, which makes "find the best run and do something with it" a three-line script rather than a project.

✓ **What comparison gives you**

- A diff of parameters across selected runs
- Overlaid metric curves by step
- Parallel coordinates to see which parameter drives the metric
- A queryable table you can drive from Python

✗ **What it cannot do for you**

- Compare numbers you only `print()` ed
- Compare parameters you never logged
- Distinguish runs you never named or tagged
- Explain a difference caused by unlogged data changes



runs navigable. Set them in the script rather than by hand, because the run you forget to label is always the one you need to find later.

► TRY IT

1. Run one script four times with different depths, each with a descriptive `run_name` .
2. Add the depth and your metric as columns and sort by the metric.
3. Select all four and open the Compare view, then the Chart view's parallel coordinates.
4. Reproduce the ranking in Python with `mlflow.search_runs` and a `filter_string` .

Expected result – parallel coordinates in step three is the underused one: with four runs it looks decorative, and with forty it tells you immediately which parameter matters. Step four matters because that same query is what a CI promotion check runs.

PYTHON

Copy

```
# 1. At log time – the most common
mlflow.sklearn.log_model(model, name="model", registered_model_name="churn-classifier")

# 2. From an existing run afterwards
mlflow.register_model("runs:/a1b2c3d4e5f64718/model", "churn-classifier")

# 3. Through the client, with explicit control
from mlflow import MlflowClient
client = MlflowClient()
```



```
source="runs:/a1b2c3d4e5f64718/model",  
run_id="a1b2c3d4e5f64718",  
)
```

Each registration creates a **numbered version** (version 1, 2, 3) which is immutable in content and permanently linked back to the run that produced it.

Then you point at a version by **alias** rather than by number, which is the part that makes deployment sane:

PYTHON

Copy

```
from mlflow import MlflowClient  
client = MlflowClient()  
  
client.set_registered_model_alias("churn-classifier", "champion", version=7)  
client.set_registered_model_alias("churn-classifier", "challenger", version=8)  
  
# Consumers never mention a version number  
model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")
```

1 Logged

A run produces a model artifact with a signature and an environment.

2 Registered

It becomes version N of a named registered model, linked to its run.

3 Evaluated

A separate job scores it and records the result, often as a tag on the version.

4 Aliased

`champion` moves to that version. Every consumer follows the alias.

5 Superseded

A newer version takes the alias; the old one stays queryable by number forever.



► TRY IT

1. Register two models from two different runs and confirm you get versions 1 and 2.
2. Set the `champion` alias to version 1, then load `models:/name@champion` and print a prediction.
3. Move the alias to version 2 and rerun the exact same loading code. Nothing in your code changed.
4. From a registered version in the UI, click through to the run that produced it and find its parameters.

Expected result – step three is the entire point of a registry: a deployment changed without a code change, a redeploy, or a file path. Step four is the audit path, weights to run to parameters to commit, and it should take you seconds.

Loading and serving: batch, REST, container



In a **batch job**, which is the most common and the easiest:

batch_score.py

Copy

```
import mlflow
import pandas as pd

model = mlflow.pyfunc.load_model("models:/churn-classifier@champion")
frame = pd.read_parquet("data/today.parquet")
frame["score"] = model.predict(frame[model.metadata.get_input_schema().input_names()])
frame.to_parquet("out/scored.parquet")
```



MLOps MENA
COMMUNITY

العربية



Join the Community ↗





Evaluating with `mlflow.evaluate`

Logging your own metrics is fine. For standard tasks there is a built-in evaluator that produces a consistent, comparable set, plus plots and an explainability summary, from one call.

PYTHON

Copy

```
import mlflow
from mlflow.models import infer_signature

with mlflow.start_run(run_name="rf + eval"):
    model.fit(X_train, y_train)
    signature = infer_signature(X_train, model.predict(X_train))
    info = mlflow.sklearn.log_model(model, name="model", signature=signature)

    eval_data = X_test.copy()
    eval_data["label"] = y_test

    result = mlflow.evaluate(
        model=info.model_uri,
        data=eval_data,
        targets="label",
        model_type="classifier",
        evaluators=["default"],
    )
    print(result.metrics["roc_auc"], result.metrics["f1_score"])
```



MLOps MENA
COMMUNITY

العربية



Join the Community ↗





Recorded automatically	Tag or field
Source file or notebook	<code>mlflow.source.name</code>
Git commit, if run inside a repository	<code>mlflow.source.git.commit</code>
Whether the tree was dirty	<code>mlflow.source.git.repoURL</code> , plus a dirty warning
User	<code>mlflow.user</code>
Run type	<code>mlflow.source.type</code> : <code>LOCAL</code> , <code>PROJECT</code> , <code>JOB</code> , <code>NOTEBOOK</code>
Model dependencies	<code>requirements.txt</code> inside the model directory

Not recorded unless you do it	How
The data you trained on	<code>mlflow.log_input</code> with a dataset, or a tag naming a version
Random seeds	Log them as parameters and set them
Uncommitted code	MLflow records the commit, not a diff : commit before real runs
System packages, CUDA	Use a container; the pip list is not the whole environment



feature definitions

Log the feature-set version as a param or tag

logging the dataset, so the run names its input

Copy

```
import mlflow.data
import pandas as pd

frame = pd.read_parquet("data/train_v3.parquet")
dataset = mlflow.data.from_pandas(frame, source="s3://bucket/data/train_v3.parquet", name="train_v3")

with mlflow.start_run():
    mlflow.log_input(dataset, context="training")
    mlflow.log_params({"seed": 42, "feature_set": "v3"})
    ...
```



BASH

Copy

```
# Run locally, with overrides  
mlflow run . -P max_depth=8 -P n_estimators=400
```



```
# Run straight from git, at a pinned commit
mlflow run https://github.com/org/repo.git --version 3f9a1c2 -P max_depth=8

# Skip environment creation when you know the current env is right
mlflow run . --env-manager local
```

Field	Purpose
<code>name</code>	Shows up on the run
<code>python_env</code> / <code>conda_env</code> / <code>docker_env</code>	How the environment is built
<code>entry_points</code>	Named commands; <code>main</code> is the default
<code>parameters</code>	Typed, with defaults: overridable with <code>-P</code>
<code>command</code>	The shell command, with <code>{param}</code> substitution

A run launched this way gets `mlflow.source.type = PROJECT` and records the entry point and its parameters, so the run itself tells you the command that produced it.



Reading a failed run, in order

Debugging MLflow has an order, and following it beats guessing.

01 Check the run's status and whether it exists at all

A run stuck in `RUNNING` means the process died without closing it. No run at all means the tracking URI was wrong and you wrote to a local `mlruns/` somewhere.

02 Confirm the tracking URI and experiment

`mlflow.get_tracking_uri()` and `mlflow.get_experiment_by_name(...)`. Runs landing in the wrong place is the most common "MLflow is broken".

03 Read the run's tags

Source file, git commit, and user. A surprising commit explains a lot.

04 Read the params, not your code

Especially with autolog, the recorded parameters are what ran.

05 Open the model's `MLmodel` and `requirements.txt`

Serving and loading failures are nearly always environment mismatches, and this is where the truth is.

06 Reload the model in a clean environment

`mlflow models predict -m ... --env-manager virtualenv` reproduces the serving path without standing up a server.



MLOps MENA
COMMUNITY

العربية



Join the Community ↗





MLOps MENA
COMMUNITY

العربية



Join the Community ↗





MLOps MENA
COMMUNITY

العربية



Join the Community ↗





MLOps MENA
COMMUNITY

العربية



Join the Community ↗





A named experiment, not <code>default</code>	The four components
Autolog for framework params, explicit calls for yours	Autologging
The dataset logged with <code>log_input</code>	Reproducibility
with <code>mlflow.start_run()</code> and a descriptive <code>run_name</code>	Your first tracked run
Tags set in code, never by hand	Comparing runs
A signature inferred and logged	The MLflow Model format
Registered at log time with <code>registered_model_name</code>	The Model Registry
<code>mlflow.evaluate</code> so every run is comparable	Evaluating models
Promotion by moving an alias , with a margin	The Model Registry
An <code>MLproject</code> entry point, so a run is one command	MLflow Projects

► TRY IT: THE ONE THAT MATTERS

1. Take this structure into a project you work on, adapting the training body to your own code.
2. Get one run green with autolog, a custom metric, a signature, a registered model, and an evaluation.
3. Run it twice with different parameters and pick a winner from the experiment table.
4. Promote the winner by moving the `champion` alias, then serve `models:/name@champion` locally and call it with `curl`.
5. Move the alias to the other version and call the endpoint again without changing a line of code.

Expected result – step five is the acceptance test: a deployment that changed by moving a pointer. If anything in your loop required editing a path or a version number, that is the spot to fix. It is exactly where production drift comes from.



MLOps MENA
COMMUNITY

العربية



Join the Community ↗





dataset tracking and lineage, the registry as a promotion workflow with webhooks and CI, `mlflow.evaluate` with custom metrics and validation thresholds, MLflow Tracing and LLM evaluation, deployment targets and the deployments API, autologging internals, the client API and bulk operations, and the CI patterns that make a training run a pull-request check.

Senior then covers what you own when MLflow is your team's platform: the self-hosted deployment and its stores, authentication and multi-tenancy, artifact-store access control and credential handling, backend database scaling and what breaks first, backup and upgrade procedure, cost and retention policy for artifacts, lineage and audit for regulated work, promotion gates and approval trails, incident playbooks, and where MLflow stops and a feature store, an orchestrator, or a dedicated serving platform begins.

NEXT UP

Beginner · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.





MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**